

TDA - Gestor de Tareas de Servidor

Profesor: Sebastián I. Alvarado Aceituno
Mail: sebastian.alvarado2@mail.udp.cl

Descripción del problema:

En este laboratorio, modelarás el motor lógico de un centro de cómputo utilizando Tipos de Datos Abstractos (TDA) de C++. Tu objetivo es implementar una clase **GestorServidor** que administre servidores, tareas encoladas, historiales de ejecución y el registro de tiempos de procesamiento mediante el uso de **Vectores, Colas, Pilas y Mapas**.

Requisitos de implementación:

Implementar la clase **GestorServidor** con atributos **privados** y los métodos públicos necesarios para que el programa de prueba funcione. La distribución de puntaje (60 puntos en total) es la siguiente:

1. **Uso de `std::vector`** (10 puntos): Mantén un registro de los nombres de los servidores activos. Implementa un método `agregarServidor(string)`.
2. **Uso de `std::map`** (15 puntos): Utiliza un diccionario (clave-valor) que asocie el nombre de cada tarea (clave) con su tiempo de procesamiento en milisegundos (valor).
3. **Uso de `std::queue`** (10 puntos): Crea una cola de tareas pendientes (almacenando los nombres de las tareas). Implementa un método `agregarTarea(string, int)` que encole la tarea y simultáneamente guarde su tiempo en el mapa.
4. **Uso de `std::stack`** (10 puntos): Mantén un historial de las tareas procesadas. Implementa el método `procesarSiguienteTarea()` que extraiga la tarea de la cola, la "procese" (imprimiendo un mensaje con su tiempo obtenido del mapa) y la empuje a la pila del historial.
5. **Método `mostrarEstado()`** (15 puntos): Este método debe imprimir correctamente los servidores activos iterando sobre el vector, mostrar cuántas tareas quedan en la cola y cuál es la siguiente, vaciar la pila para mostrar el historial ordenado desde la última tarea procesada a la primera, e iterar sobre el mapa para imprimir el registro de tiempos.

Código de prueba (main.cpp):

Se te proporciona la función `main`. **No debes modificar este código**. Tu tarea es escribir las librerías necesarias y la clase **GestorServidor** para generar la salida esperada.

```
#include <iostream>
#include <string>
// TODO: Incluye las librerías TDA necesarias aquí

using namespace std;

// Escribe aquí tu clase GestorServidor

int main() {
    GestorServidor miGestor;

    // 1. Agregar servidores activos
    miGestor.agregarServidor("Servidor_Alfa");
    miGestor.agregarServidor("Servidor_Beta");

    // 2. Agregar tareas a la cola y registrar su tiempo estimado (ms)
    miGestor.agregarTarea("Procesar_Datos", 150);
    miGestor.agregarTarea("Entrenar_Modelo", 450);
    miGestor.agregarTarea("Generar_Reporte", 50);

    cout << "---- Iniciando Procesamiento ----" << endl;

    // 3. Procesar tareas de la cola y pasarlas al historial
```

```
miGestor.procesarSiguienteTarea();
miGestor.procesarSiguienteTarea();

cout << "\n--- Estado Actual del Gestor ---" << endl;

// 4. Mostrar el estado de todas las estructuras
miGestor.mostrarEstado();

return 0;
}
```

Salida esperada:

Si diseñas tu clase correctamente, al ejecutar el programa deberías ver exactamente esto en la consola:

```
--- Iniciando Procesamiento ---
Procesando tarea 'Procesar_Datos' (150 ms)... Completada.
Procesando tarea 'Entrenar_Modelo' (450 ms)... Completada.

--- Estado Actual del Gestor ---
Servidores Activos:
- Servidor_Alfa
- Servidor_Beta

Tareas pendientes en la cola: 1
Siguiente tarea en cola: Generar_Reporte

Historial de tareas completadas (Ultima a primera):
1. Entrenar_Modelo
2. Procesar_Datos

Registro de tiempos (Orden alfabetico automatico del map):
- Entrenar_Modelo: 450 ms
- Generar_Reporte: 50 ms
- Procesar_Datos: 150 ms
```